

Standardization Bot — Getting Access

This guide gets you from zero to using the Standardization Bot. Pick the part that matches what you want to do — most people only need **Part 1** or **Part 2**. For what the bot's results mean once you have access, see the User Guide.

Contact for everything: Max Fan (max.fan@paramount.com).

Read this first: the bot runs on one laptop right now

The bot is a proof of concept. It is **not hosted in the cloud yet** — it runs locally on the maintainer's machine. That means **the bot only works while that machine is on, awake, and has the bot running**.

If the bot ever seems broken, the cause is almost always that it isn't running. Nothing is wrong on your end — message Max and it will be back in a couple of minutes.

How to tell: in Slack you'll see “/review-spec failed because the app did not respond”. In Google Docs you'll see “HTTP 404 ... endpoint ... is offline (ERR_NGROK_3200)” or a 502 page.

Which part do I need?

I want to...	Go to	What you need
Check a spec or look up vendors in Slack	Part 1	Nothing — just Slack
Get a “Spec check” menu inside my Google Doc	Part 2	A copy of the spec template
Start the bot / keep it running (maintainer)	Part 3	The project folder on your Mac
Set up my own independent copy (developer)	Part 4	The code + your own credentials

Part 1 — Use the bot in Slack

Who this is for: anyone reviewing or writing tracking specs. No setup, tokens, or permissions needed.

Heads-up

While we're still running the bot locally, it must be **running on a machine** (currently Max's laptop) for Slack to get a reply. If a command fails with “the app did not respond”, the bot is simply off — ping Max.

Steps

1. Open Slack (the Paramount workspace you already use).
2. In the search bar, type **privacy-specs-bot** and open a direct message (DM) with it. If you can't find it, ask Max to confirm the app is installed.
3. Send the bot one of the following:
 - **A spec file** — drag a **PDF** or **.docx** into the DM. You get a spec doc check back.
 - **A vendor list** — type something like **Conviva, Branch, Adobe Analytics**. You get each vendor's OneTrust category and profile rules back.
4. Wait for the reply in the same DM (usually under a minute).

Slash commands (optional)

These work in any channel where the bot is a member. To add it to a channel, type `/invite @privacy-specs-bot` there. Or just use them in the DM.

Command	What it does
<code>/review-spec <paste spec text></code>	Spec doc check on pasted text
<code>/vendor-check Conviva, Kochava</code>	Vendor privacy lookup
<code>/spec-log</code>	Lists recent reviews (who, when, what)
<code>@privacy-specs-bot</code>	Shows help text

Tips

- Only PDF and .docx are accepted. For a native Google Doc, use **File → Download → Microsoft Word (.docx)** first, or use Part 2 instead.
- The bot reads tables in .docx files, so specs with variable tables work well.
- If nothing comes back at all, the bot isn't running — see the box at the top.

Part 2 — Use the spec check inside Google Docs

Who this is for: spec authors who want a **Spec check** menu right inside their spec doc, so they can review without leaving Google Docs.

Heads-up

The Google Docs menu talks to the same bot. While we're still running it locally, the review service must be **running on a machine** (currently Max's laptop) and its tunnel must be up. If you see “endpoint ... is offline (ERR_NGROK_3200)”, a 404, or a 502, the service is off — ping Max. Nothing on your end is wrong.

Easiest way: copy a doc that already has the script

The spec check is an Apps Script attached to the document. **When you make a copy of a doc that has the script, the copy automatically includes it** — no code, no setup.

1. Open the spec template (or any spec doc that already has the **Spec check** menu): <link to the spec template doc – to be added>
2. Go to **File → Make a copy**, name it, and choose where to save it.
3. Open your copy. After a few seconds a **Spec check** menu appears in the menu bar (next to Help). If it doesn't, reload the page.
4. Write your spec as usual.
5. When you're ready, click **Spec check → Review this spec**.
6. **First time only:** Google asks you to authorize the script.
 - Click **Continue**, choose your Paramount account.
 - If you see “Google hasn't verified this app”, click **Advanced → Go to ... (unsafe)**. It's our own internal script.
 - Click **Allow**. This is asked once per person, per document.
7. Results open in a sidebar on the right. Fix what's flagged and run it again as often as you like.

Adding the script to an existing doc (manual)

Only needed if your doc was **not** copied from the template. Takes about two minutes.

1. Open the doc → **Extensions → Apps Script**. A script editor opens in a new tab.
2. Delete the placeholder code and paste the script below.
3. Check the **REVIEW_URL** line at the top. It should be exactly: <https://garter-thermos-delirious.ngrok-free.dev/review>
4. Click the **Save** icon (or Ctrl/Cmd + S).
5. Go back to the doc and **reload** it. The **Spec check** menu appears.
6. Run **Spec check → Review this spec** and accept the one-time authorization prompt (see above).

```
const REVIEW_URL = 'https://garter-thermos-delirious.ngrok-free.dev/review';

function onOpen() {
  DocumentApp.getUi()
    .createMenu('Spec check')
    .addItem('Review this spec', 'reviewSpec')
    .addToUi();
}

function reviewSpec() {
  const text = DocumentApp.getActiveDocument().getBody().getText();
  const resp = UrlFetchApp.fetch(REVIEW_URL, {
```

```

method: 'post',
contentType: 'application/json',
payload: JSON.stringify({ text: text }),
muteHttpExceptions: true,
});
const findings = JSON.parse(resp.getContentText()).findings ||
  resp.getContentText();
const html = HtmlService
  .createHtmlOutput('<pre style="white-space:pre-wrap">' + findings +
'</pre>')
  .setTitle('Spec check');
DocumentApp.getUi().showSidebar(html);
}

```

If something goes wrong in Google Docs

What you see	What it means	What to do
No Spec check menu	Script hasn't loaded, or the doc has no script	Reload the page. If still missing, copy the template or add the script manually (above).
"endpoint ... is offline" / ERR_NGROK_3200 / HTTP 404	The tunnel on the host machine isn't running	Tell Max. Nothing to fix on your side.
HTTP 502	Tunnel is up but the review server isn't	Tell Max.
Authorization prompt every time	Normal for the very first run only	Accept once; it shouldn't ask again.
Review takes a long time	First request after a restart is slower	Wait up to a minute and try again.

Part 3 — Starting the bot (maintainer)

Who this is for: whoever is hosting the bot (today: Max). Until the bot is deployed to a server, this is the checklist to bring it up each day or after a laptop restart. Three things have to be running at the same time, each in its own Terminal window:

#	Piece	What it powers	You know it's up when...
1	<code>python server.py</code>	The review engine + Google Docs endpoint	It prints "... reference specs loaded. Serving."
2	<code>ngrok</code>	The public URL the Google Docs script calls	It shows "Session Status: online"
3	<code>python app.py</code>	The Slack bot	It prints "⚡ Bolt app is running!"

Step by step

1. Open **Terminal** and go to the project folder:

```
cd "/Users/fanm0601/github/Standardization Bot"
```

2. **Window 1 — start the review server.** It loads every prior spec from Google Drive first, so expect **2–5 minutes** of “Loading reference specs...” before it says “Serving.” That's normal.

```
source .venv/bin/activate && python server.py
```

3. **Window 2 — start the tunnel.** Open a new Terminal window/tab (Cmd + T) and run:

```
ngrok http 8080 --url=garter-thermos-delirious.ngrok-free.dev
```

This uses our reserved ngrok domain, so the URL in the Google Docs script **never changes**. Don't run plain **ngrok http 8080** — that gives a random URL and breaks every doc's script.

4. **Window 3 — start the Slack bot.** New window/tab again, then:

```
cd "/Users/fanm0601/github/Standardization Bot" && source .venv/bin/activate && python app.py
```

5. **Check it works.** In a fourth tab (or a browser) open the health URL — it should say **ok**:

```
curl https://garter-thermos-delirious.ngrok-free.dev/health
```

Then DM the bot in Slack, or run **Spec check → Review this spec** in a Google Doc.

Keep it running

Leave all three Terminal windows open. **Closing a window, quitting Terminal, or letting the laptop sleep or shut down stops the bot.** In System Settings → Displays → Advanced (or Battery), prevent automatic sleeping while plugged in.

To stop a piece, click its window and press **Ctrl + C**. To restart, re-run the same command.

New specs added to the Drive folder aren't picked up automatically. Restart **server.py** and **app.py** to reload them.

Troubleshooting (maintainer)

Symptom	Cause	Fix
app.py crashes with not_authenticated / “token is invalid”	SLACK_BOT_TOKEN or SLACK_APP_TOKEN is blank or wrong in .env	Paste the tokens from api.slack.com/apps (Bot token xoxb-... under OAuth & Permissions; App token xapp-... under Basic Information → App-Level Tokens). Restart.
“Address already in use” when starting server.py	An old copy is still running	pkill -f server.py then start it again.

Symptom	Cause	Fix
Google Docs shows ERR_NGROK_3200 / 404	ngrok isn't running	Start Window 2.
Google Docs shows 502	ngrok is up but <code>server.py</code> isn't (or is still loading)	Start Window 1, or wait for "Serving."
"Failed to load Drive item ... 403 Forbidden" at startup	That Google Doc isn't shared with the bot's service account	Share the doc (Viewer) with <code>tracking-spec-review@i-digmops.iam.gserviceaccount.com</code> . Other specs still load fine.
"Read timed out" for one Drive item at startup	Transient network hiccup	Ignore, or restart to retry.
Slack says "the app did not respond"	<code>app.py</code> isn't running	Start Window 3.
Certificate / SSL errors on the corporate VPN	TLS interception	<code>.env</code> already points <code>REQUESTS_CA_BUNDLE</code> and <code>SSL_CERT_FILE</code> at <code>ca-bundle.pem</code> . Rebuild it if certificates were rotated (see Part 4).

Making it always-on (next step)

Running on a laptop is the current limitation behind every "it isn't responding" report. To make access public and permanent, the plan is to:

1. Deploy `server.py` and `app.py` to an always-on host (e.g. a small Cloud Run or Compute Engine service in the `i-digmops` GCP project) with the same `.env` values stored as secrets.
2. Point the Google Docs script's `REVIEW_URL` at the hosted URL (or keep the reserved ngrok domain fronting it), and update the template doc once.
3. Retire the laptop setup in this Part.

Part 4 — Run your own copy (developers)

Who this is for: developers who want their own independent instance or want to change the code. Everything is set up under **your own accounts** — nothing to request from Max. Your instance has its own Slack bot and its own review log.

One-time setup

1. Get the code: <https://github.com/MaxFan-p/Intern-Project>
2. Install Python 3.10+ (3.14 is what the maintainer runs), then in the project folder:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

3. Create a `.env` file in the project folder with the credentials in the table below (see `config.py` for every setting and default).
4. Start it exactly as in Part 3 (`server.py` + `ngrok` for Google Docs, `app.py` for Slack).

Credentials you need

Credential	What it's for	How to get your own
<code>SLACK_BOT_TOKEN</code> / <code>SLACK_APP_TOKEN</code>	Your bot's Slack identity	Create your own Slack app at api.slack.com/apps (Socket Mode on; scopes <code>chat:write</code> , <code>commands</code> , <code>files:read</code> , <code>app_mentions:read</code> , <code>im:history</code> ; slash commands <code>/review-spec</code> , <code>/vendor-check</code> , <code>/spec-log</code>). Install it to the workspace. Bot token from OAuth & Permissions, App-Level token (<code>connections:write</code>) from Basic Information.
Google Cloud ADC	Vertex AI (Gemini) calls	<code>gcloud auth application-default login</code> with your Paramount account. Set <code>GOOGLE_CLOUD_PROJECT</code> to a GCP project with the Vertex AI API enabled.
<code>CONFLUENCE_API_TOKEN</code> (+ <code>CONFLUENCE_EMAIL</code> , <code>CONFLUENCE_BASE_URL</code> , <code>CONFLUENCE_PAGE_ID</code>)	OneTrust vendor page lookup	Generate a token at id.atlassian.com/manage-profile/security/api-tokens . Self-service, as long as your account can view the OneTrust 3rd Party Vendors Confluence page (ID 65778526).
<code>GDRIVE_SERVICE_ACCOUNT_KEY</code>	Reading prior specs from the "P+ MON Active Specs" Drive folder	Create a service account in your GCP project (IAM & Admin → Service Accounts), download its JSON key, set the path in <code>.env</code> . Ask a manager of the shared drive to share the folder (Viewer) with the service account's <code>client_email</code> . Leave <code>GDRIVE_FOLDER_NAME=P+ MON Active Specs</code> or set <code>GDRIVE_FOLDER_ID</code> from the folder URL. If Drive is unreachable, the bot falls back to a local <code>references/</code> folder.
<code>REQUESTS_CA_BUNDLE</code> / <code>SSL_CERT_FILE</code>	Google API calls from the corporate VPN	Build a combined CA bundle and point both variables at it: <code>cat \$(python -c 'import certifi;print(certifi.where())') <(security find-certificate -a -p /Library/Keychains/System.keychain) > ca-bundle.pem</code>

Credential	What it's for	How to get your own
ngrok account (free)	Public URL for the Google Docs sidebar	Sign up at ngrok.com, install ngrok, run <code>ngrok config add-authtoken <token></code> . Claim your one free static domain in the ngrok dashboard and use it with <code>--url=</code> .

Don't reuse someone else's Slack tokens

If two machines run the bot with the same tokens, Slack splits events between them and each instance sees only some messages — it looks like the bot randomly ignores people. **One Slack app per running instance.**

Quick reference

I want to...	I need...	Ask
Check specs in Slack	Nothing — DM privacy-specs-bot	Max if the bot is unresponsive
Spec check menu in my Google Doc	A copy of the spec template (File → Make a copy)	Max for the template link
Bring the bot back up	The three Terminal windows in Part 3	—
Run my own instance	The code + my own credentials (Part 4)	Nobody — all self-service